

A real dataset, and a test score that lies

891 real passenger records with gaps in them. Commit data leakage on purpose and watch the test score reach a perfect, meaningless 100%. Then complete a pipeline that cannot leak.

Guided lab

20 minutes

pandas + scikit-learn

Split first

Before you start Setup · given 1 · the baseline first 2 · leak on purpose · given

3 · complete the pipeline 4 · the quiet leak · given Reflection If you finish early

Before you start

Open the provided `lab14.ipynb` with the `data/` folder beside it. The dataset ships partly cleaned: this session is about leakage and order of operations, not data janitorial work.



By the end of this lab you can

- Inspect missing values and compute a majority-class baseline.
- Recognise target leakage from a suspiciously perfect score.
- Build a preprocessing pipeline that is fitted on training data only.
- Explain why the split has to happen before imputation and scaling.

Setup · given

The Titanic passenger list — a real historical record, widely used for teaching because its gaps are real.

`survived` is the target.

given

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("data/titanic_partly_cleaned.csv")
print(df.head(3).to_string())
print("\nmissing values per column:\n" + df.isna().sum().to_string())
```

what it should print

OUTPUT

	survived	pclass	sex	age	sibsp	parch	fare	embarked	deck	alive
0	0	3	male	22.0	1	0	7.2500	S	NaN	no
1	1	1	female	38.0	1	0	71.2833	C	C	yes
2	1	3	female	26.0	0	0	7.9250	S	NaN	yes

missing values per column:

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
deck	688
alive	0

TASK 1 · 3 MINUTES

the baseline first

1

Before any model: what accuracy do you get by predicting the more common outcome for everyone?

your turn

PYTHON

```
baseline = None      # TODO: one line using df["survived"]
print("baseline accuracy:", round(baseline, 3))
```

one version that works

PYTHON

```
baseline = df["survived"].value_counts(normalize=True).max()
print("baseline accuracy:", round(baseline, 3))
```

what it should print

OUTPUT

```
baseline accuracy: 0.616
```

TASK 2 · 4 MINUTES

leak on purpose · given

2

Run this. It uses **every** column as a feature, with a quick one-hot encoding, and scores on a held-out test set.

given

PYTHON

```
X_all = pd.get_dummies(df.drop(columns=["survived", "deck"]), dtype=float).fillna(0)
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(X_all, df["survived"], test_size=0.25,
random_state=7, stratify=df["survived"])
leaky = LogisticRegression(max_iter=2000).fit(Xa_tr, ya_tr)
print("test accuracy:", round(leaky.score(Xa_te, ya_te), 3))
```

what it should print

OUTPUT

test accuracy: 1.0



🔴 **A perfect score on real data is a symptom, not a result.** Look at the column list in the setup cell. Which column gives the answer away, and why would it never be available at the moment the prediction is needed?

The leaking column, and why:

TASK 3 · 9 MINUTES · THE MAIN TASK

complete the pipeline

3

Three blanks. The structure is written: numeric columns get median imputation and scaling, categorical columns get most-frequent imputation and one-hot encoding, and the whole thing is fitted **only** on the training rows.



🚫 `deck` is missing for 77% of passengers and `alive` is the leak. Leave both out of your column lists.

```
numeric      = None      # TODO blank 1: a list of the numeric feature columns
categorical  = None      # TODO blank 2: a list of the categorical feature columns

X = df[numeric + categorical]
y = df["survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=7,
stratify=y)

prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale",
StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")), ("onehot",
OneHotEncoder(handle_unknown="ignore"))]), categorical),
])
pipe = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=2000))])

None          # TODO blank 3: fit the whole pipeline – on which rows?
test_acc = pipe.score(X_test, y_test)
print("honest test accuracy:", round(test_acc, 3), "    baseline:", round(baseline, 3))
assert 0.75 < test_acc < 0.95, "below 0.75: check the columns; above 0.95: something is
leaking"
```

one version that works

PYTHON

```
numeric      = ["pclass", "age", "sibsp", "parch", "fare"]
categorical = ["sex", "embarked"]
X = df[numeric + categorical]
y = df["survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=7, stratify=y)
prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale",
StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")), ("onehot",
OneHotEncoder(handle_unknown="ignore"))]), categorical),
])
pipe = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=2000))])
pipe.fit(X_train, y_train)
test_acc = pipe.score(X_test, y_test)
print("honest test accuracy:", round(test_acc, 3), "    baseline:", round(baseline, 3))
```

what it should print

OUTPUT

```
honest test accuracy: 0.771    baseline: 0.616
```

TASK 4 • 4 MINUTES

the quiet leak • given

4

The `alive` column was a loud leak. This one is quiet: the median and the scaling statistics are computed on **all** rows, test rows included, before the split.

given

PYTHON

```
from sklearn.model_selection import cross_val_score
from sklearn.base import clone

X_leaky = X.copy()
X_leaky[numeric] = SimpleImputer(strategy="median").fit_transform(X_leaky[numeric]) #
# fitted on every row
X_leaky[numeric] = StandardScaler().fit_transform(X_leaky[numeric]) #
# fitted on every row
quiet = cross_val_score(clone(pipe), X_leaky, y, cv=10).mean()
honest = cross_val_score(clone(pipe), X, y, cv=10).mean()
print("preprocess-then-split:", round(quiet, 4), "    split-then-preprocess:", round(honest,
4))
```

what it should print

OUTPUT

```
preprocess-then-split: 0.7912    split-then-preprocess: 0.7901
```

Here the difference is small, and that is the danger: a quiet leak never looks wrong. In one sentence — why is the second number the only one you are allowed to report, even when the first is barely different?

Your answer:

Reflection



Write it in the notebook, in your own words

Session 4 ended on SNIA 2030 and why this course is taught in Grand-Bassam. Name one Ivorian dataset you would want a model trained on — health, agriculture, transport — and one column in it that could leak the answer the way `alive` did.

Your answer:

If you finish early

- Put `deck` back in as a categorical column, with missing values filled by a constant `"unknown"` instead of the most frequent value. Does the honest accuracy change? Is "deck unknown" itself informative?
- Print the confusion matrix of your pipeline on the test set. Friday's worksheet is about exactly this table.

your turn

PYTHON

```
# TODO (optional extensions A, B)
```

CSC 4315 — Artificial Intelligence · Lab 14 · accompanies session 20 · Fall 2026

Prof. Guillaume Sanje Mpacko, Ph.D. · School of Science and Engineering · International University of Grand-Bassam